Preliminary Phase - Collecting Info

September 4, 2024 6:59 PM

This is the first general guideline, this is subject to change

Goal

The general goal is a medium complexity CPU simulator that demonstrates the following

1) fetch → decode → execute cycles
2) register based computation
3) Some basic arithmetic & memory operations
4) simple branching → needs more research

} might be expanded on later

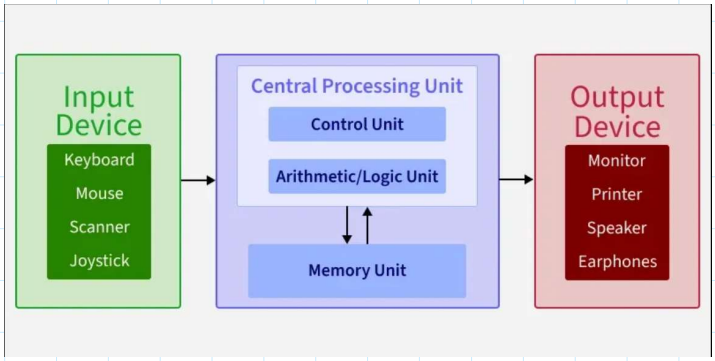
I am starting with python first rather than verilog since its easier to debug and understand as of now my knowledge is limited, so this is probably best.

Architecture type

Ended up going with Von-neumann Architecture over Harvard Architecture.

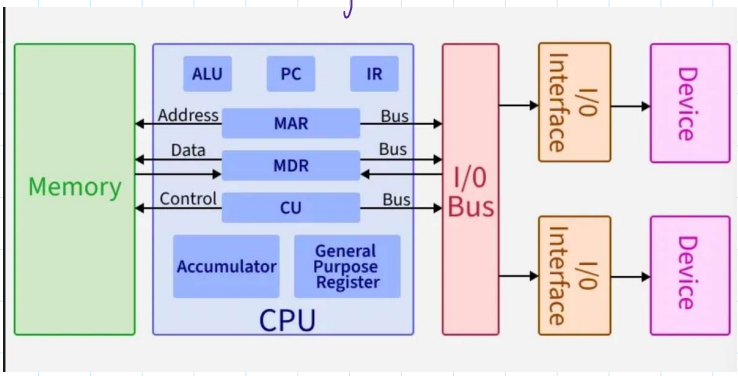
I chose this because it seemed to be easier to implement, matches most actual CPU simulator designs, and is generally better for learning

- Harvard Architecture is generally more complex and is likely unnecessary for the scope here



} Basic components

CPU: What we're building.



CU (control unit): manges how the processor works by sending control signals - Decides how data should move inside the computer, controls input & output operations, and fetches the instructions from memory for execution.

So in a nutshell its a cordinator that needs instructions & tells the rest of the CPU what to do, step by step. Does this via control signals.

ALU (Arithmetic & Logic Unit): Handles calcs & decision-making tasks. Performs arithmetic operations like adding & subtracting, logical operations like comparisons, and tasks like shifing bits in data

So basically, its the "worker" part of the CPU that actually performs operations & produces results

Registers: Fastest type of memory located inside the CPU. Temporarily store info that the processor is currently working on, making program execution & operations faster & more efficient. Are the CPU's primary working memory.

Has 6-types:

PC (Program Counter): Keep track of the address of the next instruction to be executed
IP (Instruction Register): Holds the current instruction being executed
MAR (Memory Adress Register): Temporarily holds data being transfered to or from memory
Accumulator: A register that stores intermediate results of arithmetic & logic operations.
General purpose Registers: Used for temp storage of data during processing.

Memory Data Register: Holds the data just read from memory, or the data about to be written

Flags Register (Status): Each bit represents a condition from the last ALU operation. This is needed for branching.

Bus: Communication system that transfes data, addresses, and control signals between CPU memory & I/O devices. Single bus for data & instructions.

8-Bit Architecture:

I got sick of writing, so typing it is

The bit width, in this case 8, refers to the size of data a CPU can process in a single operation; specifically the width of the data bus and general purpose registers (see previous). In this case it works with 8 bits (1 byte) at a time.

This means that:

- **Registers will hold 8-bit values 0 to 255 (unsigned) or -127 to 128 (signed, twos compliment)**
- **The ALU will perform operations on 8-bit operands (basically the inputs to an operation (or instruction)) and produces 8-bit results**
- **A single data bus transfer can only move 8 maximum bits.**

Keep in mind that this doesn't mean that everything in the system is 8-bits. The adrdess bus, instruction width, and memory size can all differ in size (as is usual in most real 8-bit CPU's) it just sets the hard cap on the amount of bits you can utilize.

Data Size & Range:
The difference between bit sizes is the ranges and the data bus width. It explodes exponentially since its 2^n. In our case using a 16-bit CPU will have a range of 0-255 or -128 to 127. So if we wanted to add two 16-bit numbers on a 8-bit CPU, id need to add low byte first, get the carry, then add the high bytes plus that carry. This is called multi-byte arithmetic. Its slower but doable.

Adress Space:
Moreover, the address space or memory is determined by the address bus width (so for addresses higher than 8-bit we have to break them up into smaller chunks, process them properly and then logically recombine them, so more work). So for 8-bit we have 256 bytes, 16 has 64 KB, 20 has 1 MB, and 32 has 4GB. 8-bit isn't actually usable in a wider sense, but for the simulation we are going to use it. Although, I saw that most actual real 8-bit CPUs use a 16-bit address bus by combining two 8-bit registers to form a 16-bit address. So maybe later I might see if I can implement this --> will likely use this.

Instruction Encoding:
There are two types of encoding

1) The single-byte Instructions (8-bit encoding). We have 8 bits total, so 4 for operands and 4 for opcode. So this means 16 possible instructions (4-bit opcode), and can only reference registers 0-15 or values 0-15. This is very limiting since you cant encode a memory address in a single byte if your address space is larger than 4 bits.
2) Multi-byte instructions --> this is common. Instead here we got something like
   a. Byte 1: opcode
   b. Byte 2: operand
   c. Byte 3: second operand

   So a example for instruction formats would be something like

   [OPCODE] - 1 byte: instructions like NOP, HLT (no operand needed)

   [OPCODE] [VALUE] - 2 bytes: LOAD_IMM 42 (load the number 42 into accumulator)

   [OPCODE] [ADDRESS] - 2 bytes: LOAD_MEM 0xA0 (load from memory address 0xA0)

   [OPCODE] [ADDR_H] [ADDR_L]- 3 bytes: if using 16-bit addresses

In any case though, I'm just going for a more simple processor so 8-bits will be the starting point but I'll likely combine two 8 bit to make 16 bit since it'd be easier to debug and deal with.

Memory Model:
Since we are using Von Neumann architecture, I am going with a single RAM array (256 bytes). This'll store both instructions and data (typical for Von Neumann architecture), and should theoretically simplify instruction fetch logic (needs testing though).

Rejected separate instructions + data memory (so the Harvard model) because with only a single bus (Von Neumann) there's no real point to splitting memory spaces. Harvard Architecture benefits from this because it can utilize parallel access (has two separate buses for instructions and data), something we don't have.

While RAM is volatile memory, this shouldn't matter here for the simulator since we are loading the program, run it, and we're done. However, in real computers they will load programs from storage (so SSD/HDD) into the RAM because RAM is significantly faster (~1000x) and because the CPU is so fast, it needs something to be able to keep up with it.

In the simulator, the CPU can only really perform operations on values in registers (because ALU works on registers, not directly on the memory). So we'll constantly need to move data between memory and registers.
Think something like this:

- LOAD: memory → register (get data so ALU can work on it)
- STORE: register → memory (save result, free up register for next operation)

Initial CPU Components - Will likely be expanded on, this is the MVP
We have a bunch for registers:
General purpose:

- R0,R1,R2,R3 -> So 8-bit. These are used for storing working data. R0 also doubles at the accumulator. Apparently It's the primary register for ALU operations, results should be stored here by default. To further explain, the accumulator is just the register that the ALU uses as its default input/output and is required regardless. So instead of having a extra separate ACC register, we can just have R0 act as the accumulator. So the same register, but has two separate roles.

Special Purpose (restating definitions from earlier and further explanations):

- The PC (Program Counter): Holds the addresses of the next instruction to fetch. This will increment after each byte fetched.
- IR (Instruction Register): This holds the opcode of the instruction being currently executed. This will be loaded during each fetch phase.
- MAR (Memory Address Register): holds the memory address the CPU wants to read from or write to.
- MDR (Memory Data Register): This holds the data just read from memory or about to be written. This will act as a buffer between the CPU and the memory.
- FLAGS (Status Register): These are individual bits set by the ALU after operations:
  - Z (Zero): set when the result is 0
  - C (Carry): Set on a unsigned overflow (so the result exceeding 255)
  - N (Negative): Set when bit 7 of result is 1 (reading right to left, this results in negative cause two's compliment)

Keep in mind that the MAR and MDR are implicitly handled in the Python implementation rather than being explicit variables, but the exist conceptually in the architecture. So what would on hardware take multiple steps we can do as something like value = memory[0x80]. Basically Python provides a abstraction for us to ignore these.
Although it might be best to make them explicit for learning purposes rather than abstract them. Will decide later during implementation.

For the ALU it'll initially support the following:

- ADD: addition (includes carry flag support)

- SUB: subtraction (has borrow/carry flag support)
- CMP - subtract WITHOUT storing the results, just updates the flags. This is needed for branching.

Later we'll see about adding in bitwise logic, invert bits, shifting, and incrementing/decrementing but that's in the future.

There were 2 other alternatives:

- ACC-based CPU, meaning that we'd only have one register for data (the accumulator). So no R0-R3 just the ACC. Any and all operations would have to go through that one register. This works but if I needed to deal with multiple values I'd need to be constantly loading and storing back to the memory because there would be only one space to store the data. This'd be a headache to work with so no.
- A larger register set (so like 8+): While real CPU's have something like 16 general purpose registers and are faster, its entirely too complex as of now and would greatly complicate things. Moreover, any test programs I make wont need this.

Instruction Format & Set:

I'll be going with a fixed length instruction format (In this case 3 bytes). Basically every instruction the CPU executes is stored as 3 bytes in memory. This is non-negotiable, regardless if you need them all it must be 3 bytes. For example HLT doesn't need arguments but will still take up 3 bytes, [opcode, 0x00, 0x00], the last two bytes are wasted padding. The upside though is that the cpu always knows the next instruction will be 3-bytes ahead, so the PC (program counter) just is doing PC+=3 after every instruction. No extra fancy stuff.

So something like this:

[what to do] [thing 1] [thing 2]

[opcode] [arg1] [arg2]

The opcode is just a number that maps to a operation, so it could look something like this:

- 0x01 = MOV
- 0x02 = ADD
- 0x03 = SUB
- 0x04 = LOAD
- etc

These arguments depend on the instruction. They could be register numbers (0 = 0 R0, 1 = R1, etc), immediate values (actual numbers like 14), or memory addresses. Basically determines how the CPU interprets the arguments.

There were 3 options to go with where the results goes:

1) ACC style: Every result goes to R0 regardless of what is specified. Foe example, ADD R1, R2 is saying R0 = R1 + R2. This matches the earlier design I was going with where R0 is the ACC. This is simple and consistent but has the adverse effect of R0 always being overwritten. This is what I'll likely choose.
2) First argument is the destination: So ADD R1, R2 means R1 = R1 + R2. The first register is both an input and where the result goes. This ends up being more flexible but a bit more complex to deal with.
3) Explicit destination as the third argument: So it'd be something like ADD R0, R1, R2 meaning R0 = R1 + R2. This is the most flexible of the three but now I need 3 register arguments plus the opcode in 3 bytes, which gets pretty tight.

Now the ISA (Instruction Set Architecture):

This is basically what the CPU can actually do. The whole point of this simulator is to gain understanding and demonstrate the fetch-decode-execute cycle doing actual work. We have a bunch of these

1) Data Movement (MOV, LOAD, STORE): MOV copies data between registers (So MOV R1, R2 means put the value of R2 into R1). However, just the registers alone aren't enough, our programs data lives in memory. LOAD gets data from memory into a register so the ALU can work on it. STORE puts the data from a register back into memory to save it. Without LOAD & STORE the CPU can only work with whatever's already in registers, so it can never really access the 256 bytes of memory I'm going to make. These are fundamental building blocks.
2) Arithmetic (ADD, SUB, CMP): ADD and SUB are as stated, they add stuff by doing arithmetic operations. CMP though is the bridge between arithmetic and branching. Basically what it does is that it subtracts two values but throws away the result; if we use this we only care about updating flags. So if we did CMP R1 R2, its saying what is the relationship between these two values. If they're equal the subtraction yields 0 and the Z flag is set, if R1 is smaller the result is negative and the N flag is set. The CPU doesn't care about storing the answer, it just remembers what the comparison result is in the flags for the next instruction to check.
3) Branching (JMP, JZ, JNZ): This is what makes CPU's more than just basic calculators. Without branching the CPU just executes instructions in a sequential manner, so instruction 0, then 1, then 2, then 3, all in a straight line forever. It won't be able to loop, skip, or make any kind of decisions. JMP address is unconditional, meaning it sets the PC (program counter) to a new address. Execution then continues from there instead of the next sequential answer. JZ address means jump to this address if the zero flag is set. So if we combine this with CMP we can get something like:
   - CMP R1, R2 → are they equal? If yes, Z flag is set
   - JZ 0x20 → if Z is set, jump to address 0x20
   - SUB R1, R2 → this only runs if they were NOT equal

   Basically a if statement at the CPU level. If we wanted to instead jump backwards to a earlier address we'd get loops. So something like:
   - Address 0x00: LOAD R0, counter → load counter value
   - Address 0x03: SUB R0, 1→ subtract 1
   - Address 0x06: STORE R0, counter→ save it back
   - Address 0x09: JNZ 0x00 → if result wasn't zero, go back to start

   That's a countdown loop. But essentially, without JMP/JZ/JNZ looping is impossible. A CPU that can't loop is pretty boring.
4) Control (HLT, NOP): HLT tells the CPU to stop. Without it, after the last actual instruction the PC (program counter) will keep incrementing and the CPU will start executing whatever random bytes are in the rest of memory. So it'll interpret data as opcodes and do random stuff until it hits a invalid opcode or the program gets killed. So every program must end with HTL to prevent this. NOP just means do nothing and move onto the next instruction. Doesn't sound that useful, but it's primarily useful for padding, timing, or leaving space to patch instructions later.

Overall the MVP instruction set should have the following:

(1) LOAD: memory to register
(2) MOV: copy between registers
(3) STORE: register to memory
(4) ADD: addition

(5) SUB: subtraction
(6) CMP: compare (sets flags without storing result)
(7) JMP: unconditional jump
(8) JZ: jump if zero flag set
(9) JNZ: jump if zero flag not set
(10) HLT: stop execution
(11) NOP: do nothing

Execution Model:
It goes Fetch -> Decode -> Execute Cycle
This is the core loop of the CPU; all instructions go through these phases. These will repeat until the CPU hits the HLT (stop) instruction.

Explaining the steps:

1) Fetch: The CPU needs to grab the instruction from memory. So, with our 3-byte fixed format this will take three steps.
   a. Read the byte at the address PC (program counter) which points to it (remember that the PC is a memory address). This is the opcode, we store it in IR (Instruction Register) and increment PC (program counter).
   b. Read the byte at the new PC (program counter). This is the first argument; store it and increment PC.
   c. Same thing, read the byte at the new PC (program counter). This would be the second argument; store it and increment PC.

   After this phase (fetching), the PC is already going to point to the next instruction. To be clear, the PC will advance during fetch, not after the execution. So the CPU will never need to say to increment the PC after executing, the next fetch should naturally read from the right place.

2) Decode: the CU (control Unit) will look at the opcode in the IR and figure out what to do. So for our simulation, because we are using Python this'd be something like a dictionary lookup or if/elif chain. It'd contain what operation it is (ADD, STORE, JMP, etc), and how to interpret the arguments (so are they register numbers, memory address, immediate values, etc. This is the opcode)

   So for example, if we had opcode 0x02 with arguments [1,2] the decoder recognize 0x02 as ADD and interpret 1 and 2 as register numbers R1 and R2.

3) Execute: This is when the CPU actually does the thing. So depending on what was decoded, a lot of different things can happen here. After the cycle will repeat from fetch using whatever the PC now points to.
   The following are some examples:
   - ADD R1, R2: ALU adds R1 + R2, result goes in R0 (accumulator), flags updated
   - STORE R0, 0x80: MAR gets 0x80, MDR gets R0's value, write to memory
   - JZ 0x20: check Z flag. If set, PC gets overwritten with 0x20 (this is why PC updates during fetch matter: JMP replaces the PC rather than incrementing it)
   - HLT: CPU stops the cycle entirely

This is where branching becomes important. Usually the PC was already advanced during the fetch, so when doing execution it will do its operation and then the fetch will grab the next instruction.

However, if we use JMP/JZ/JNZ it'll overwrite the PC during execute, meaning that the next fetch happens from a completely different address. This is the logic behind how loops and conditionals work, basically break the sequential flow by interrupting the PC.

There were two other alternatives that were dismissed.

1) Event-driven execution. This'd mean that the CPU would only run when something triggered it(so like an input event). However, real CPU's don't work like this, they are continuously cycling through fetch decode and execute as fast as the clock allows. So for that reason, I did not go with it.
2) Pipelining: Actual real modern CPU's will overlap these phases. So while one instruction is executing, the next is already being decoded, and the one after that is being fetched. This is faster, but loads more complex. I'd have to deal with a lot of headaches and as of now this is beyond my scope. Maybe further down the line when things are settled and I have a more concrete understanding of how this works I can think about implementing this.

Addressing Modes:

This'll determine how the CPU interprets the arguments given in a instruction. The same byte in arg2 could mean different things depending on the instruction (register number, literal value, or memory address). Again, the opcode is what tells the CPU which interpretation to use.

Generally there are four modes we care about:

1) Immediate: Here, the argument is the value, meaning there is no lookup needed. So, the number in arg2 is the actual data we want to use. For example, LOAD, R0, 42 will put 42 directly into R0. The 42 in the instruction bytes is the value itself.
2) Register: The argument is a register number. The CPU looks inside the register to get the actual value. So, for example, in ADD R1, R2 the 2 in arg2 is telling the CPU to go look for R2 to get the value. So if R2 contains 50, the ALU will use 50.
3) Direct (Absolute): The argument is a memory address. The CPU goes to that location in memory to grab the value. For example, in LOAD R0, 0x80 0x80 in arg2 is a memory address. The CPU will read whatever is stored at the memory location and puts it in R0.
4) Implied: The instruction doesn't need any arguments at all. The opcode itself will contain everything the CPU needs to know. So HLT & NOP have no arguments. The CPU will know what to do just from the opcode. However, we use the 3-byte fixed format, so we'd have padding in the two argument bytes.

For the MVP here, each instruction will have a fixred addressing mode. This removes ambiguity, so no mode bits. The opcode alone will tell the CPU how to read the arguments. Should look something like the following:

- MOV R1, R2: register, register
- LOAD R0, 0x80: register, direct (memory address)
- STORE R0, 0x80: register, direct (memory address)
- ADD R1, R2: register, register
- SUB R1, R2: register, register
- CMP R1, R2: register, register
- JMP 0x20: direct (address), unused
- JZ 0x20: direct (address), unused
- JNZ 0x20: direct (address), unused

- HLT: implied (both args unused)
- NOP: implied (both args unused)

A constraint of this though is that if we wanted to add immediate values (ex. ADD R0, 5 instead of ADD R0, R2), I'd need a separate opcode (something like ADD_IMM). Might add this later, but this isnt needed for the MVP. As of currently, we'll just load the value into a register first, then operate on the registers.

There is also the option of indirect addressing, where the argument is an address that contains another address (essentially a pointer). This is a bit more advanced though, so skipping it for now. Might implement later.